

Introdução ao deep learning e às redes neurais

Rafael Stutz

Aula 1 · 05/10/2026

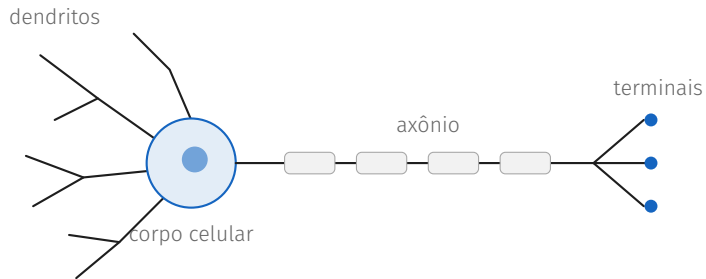
Deep Learning (DEPAI) · IASEG

Hoje

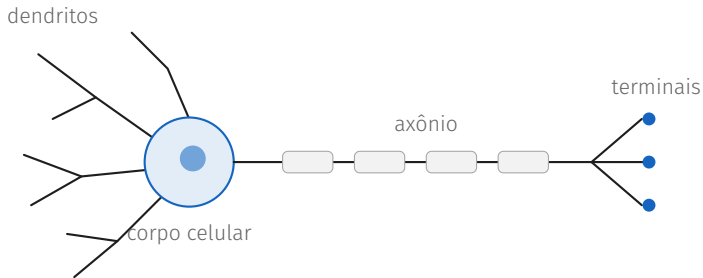
1. Planejamento do curso;
2. ML e deep learning;
3. Representações;
4. Redes neurais;
5. História do Deep Learning;
6. TensorFlow Playground;
7. Como a rede aprende;
8. Keras;
9. Fechamento.

Deixem o **playground.tensorflow.org** aberto numa aba do navegador.

O neurônio biológico

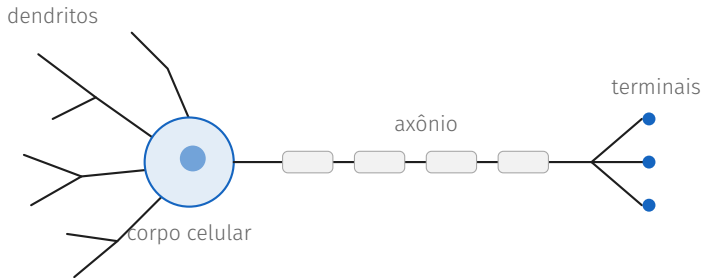


O neurônio biológico



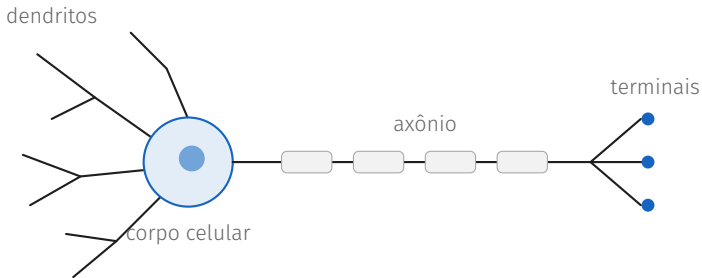
- Os **dendritos** recebem sinais de outros neurônios. Algumas ligações são fortes, outras fracas; algumas estimulam, outras inibem;

O neurônio biológico



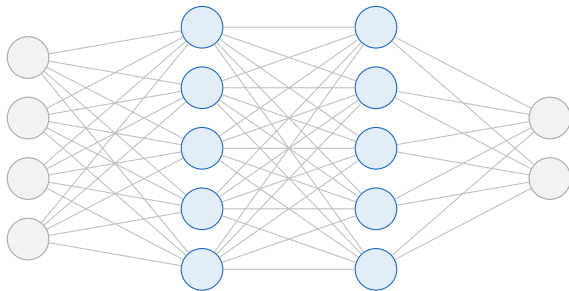
- Os **dendritos** recebem sinais de outros neurônios. Algumas ligações são fortes, outras fracas; algumas estimulam, outras inibem;
- O **corpo celular** junta esses sinais. Se o total passa de um limite, o neurônio **dispara**;

O neurônio biológico



- Os **dendritos** recebem sinais de outros neurônios. Algumas ligações são fortes, outras fracas; algumas estimulam, outras inibem;
- O **corpo celular** junta esses sinais. Se o total passa de um limite, o neurônio **dispara**;
- O disparo segue pelo **axônio** até os **terminais**, que passam o sinal para os próximos neurônios.

O que vamos estudar: redes neurais artificiais



- Cada círculo é um **neurônio artificial**: ele junta os sinais que recebe, com pesos, e decide o que passar adiante;
- A ideia veio do neurônio biológico, mas a rede **não** é um modelo do cérebro;
- Hoje vamos entender o que é um neurônio artificial, como eles se organizam em camadas e como a rede aprende.

Planejamento do curso

Cronograma

Data	Dia	Formato	Aula
05/10	seg	4h, online	1 · O curso, história, redes neurais, Keras
06/10	ter	2h, presencial	2 · Exemplos com Keras
07/10	qua	4h, presencial	3 · Curvas, sobreajuste, regularização; início das CNNs
08/10	qui	4h, presencial	4 · CNNs, aumento, transferência de aprendizado
09 a 17/10		<i>feriados e recesso: sem aula</i>	
19/10	seg	4h, presencial	5 · O que a rede olhou e mudança de distribuição
20/10	ter	4h, presencial	6 · Séries temporais
21/10	qua	4h, presencial	7 · Texto, atenção e LLMs
22/10	qui	4h, presencial	Entrega: apresentação e perguntas

Aulas a partir das 9h.

Parte	Peso	Quando
Projeto parcial	80%	relatório em 21/10; apresentação e perguntas em 22/10
Kahoot	20%	perguntas no fim de cada aula, a partir de 07/10

- São 5 rodadas: 07, 08, 19, 20 e 21/10;
- As perguntas cobrem a aula do dia, as aulas anteriores e o conteúdo de ML;
- Todas as perguntas valem o mesmo;
- Hoje e amanhã não há Kahoot.

*“Um modelo que não funciona bem é um resultado aceitável,
desde que vocês saibam dizer por quê.”*

*“Um modelo que não funciona bem é um resultado aceitável,
desde que vocês saibam dizer por quê.”*

- **Em grupo**, vocês entregam até **quarta, 21/10, 23h59** um notebook e um **relatório em 6 seções**, as mesmas do projeto de ML:
(1) objetivo da modelagem; (2) dados utilizados; (3) desempenho e comparação; (4) uso pretendido; (5) limitações; (6) restrições de uso.
- **Na apresentação, quinta, 22/10**, cada pessoa responde a perguntas do **banco de perguntas** da disciplina sobre as seções que escreveu;
- Avaliamos **como vocês chegaram ao resultado**;
- A nota **não** depende da performance do modelo.

Bases sugeridas: cada grupo escolhe uma

Base	Dado
UrbanSound8K	áudio (8.732 trechos, 10 classes)
ISP-RJ	série mensal por delegacia
Credit Default	tabela (clientes de cartão)
NYC Motor Vehicle Collisions	tabela ou série
Chicago Crimes	tabela ou série
NYPD 911 Calls for Service	tabela ou série

ML e deep learning

Onde o deep learning já está

reconhecimento facial

ChatGPT

tradução automática

detecção de disparos

geração de imagens

Claude

carros autônomos **ALPR (leitura de placas)**

diagnóstico por imagem

reconhecimento de voz

detecção de fraude

Gemini

classificação de ocorrências

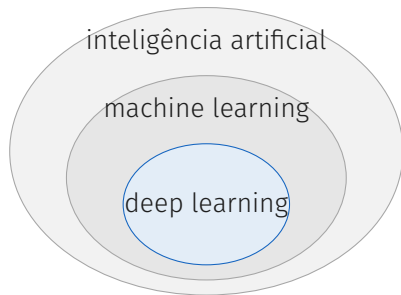
transcrição de chamadas

busca por imagem

reconhecimento de ações em vídeo

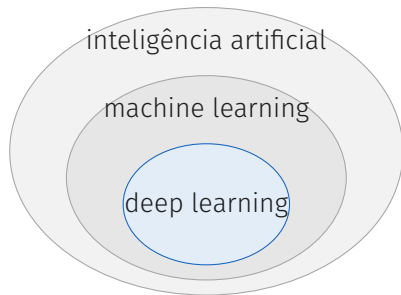
Alguém conseguiria escrever, à mão, as regras que fazem isso?

Inteligência artificial, machine learning e deep learning



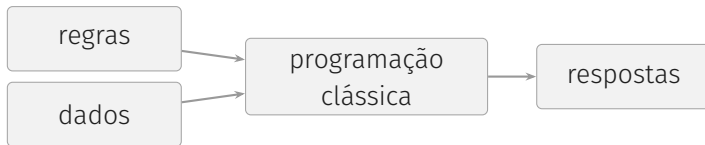
- **IA:** automatizar tarefas intelectuais feitas por pessoas. Nasce como área em 1956, num workshop em Dartmouth;
- Até os anos 1980, a IA dominante era a **IA simbólica:** programadores escrevem as regras à mão;

Inteligência artificial, machine learning e deep learning



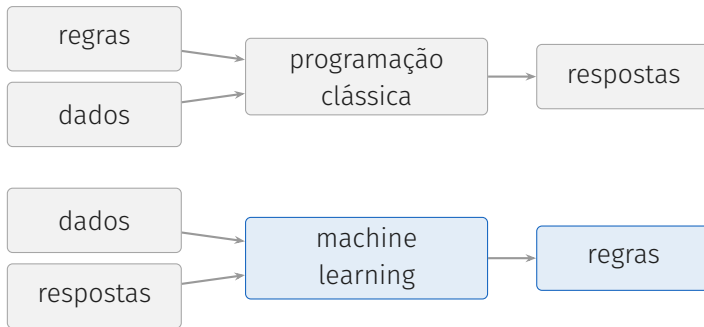
- **IA:** automatizar tarefas intelectuais feitas por pessoas. Nasce como área em 1956, num workshop em Dartmouth;
- Até os anos 1980, a IA dominante era a **IA simbólica**: programadores escrevem as regras à mão;
- Funciona em problemas bem definidos, como **xadrez**;
- Não funciona em **classificar imagens, reconhecer fala, traduzir**: ninguém consegue escrever essas regras.

Machine learning inverte a programação



Um sistema de ML é **treinado**, e não programado: ele recebe muitos exemplos e encontra a estrutura estatística que permite automatizar a tarefa.

Machine learning inverte a programação



Um sistema de ML é **treinado**, e não programado: ele recebe muitos exemplos e encontra a estrutura estatística que permite automatizar a tarefa.

O que é uma função

Uma **função** é uma regra que associa a cada **entrada** uma **saída**:

$$\text{entrada} \longrightarrow \boxed{f} \longrightarrow \text{saída}$$

O que é uma função

Uma **função** é uma regra que associa a cada **entrada** uma **saída**:

$$\text{entrada} \longrightarrow \boxed{f} \longrightarrow \text{saída}$$

Entrada	Saída	A regra f
$x = 3$	7	$f(x) = 2x + 1$
idade, classe, tarifa	chance de sobreviver	a regressão logística da Aula 3 do ML
foto de uma placa	ABC1D23	ninguém sabe escrever

O que é uma função

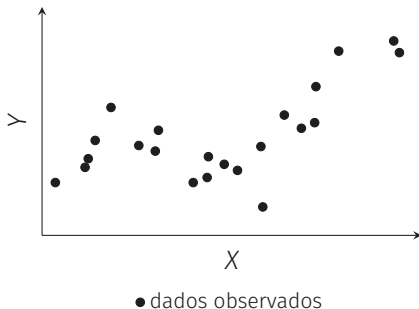
Uma **função** é uma regra que associa a cada **entrada** uma **saída**:

$$\text{entrada} \longrightarrow \boxed{f} \longrightarrow \text{saída}$$

Entrada	Saída	A regra f
$x = 3$	7	$f(x) = 2x + 1$
idade, classe, tarifa	chance de sobreviver	a regressão logística da Aula 3 do ML
foto de uma placa	ABC1D23	ninguém sabe escrever

Um modelo de ML supervisionado pode ser visto como uma função: recebe uma entrada (os dados de um caso, uma foto, um áudio) e devolve uma previsão.

O que é aprender, aqui

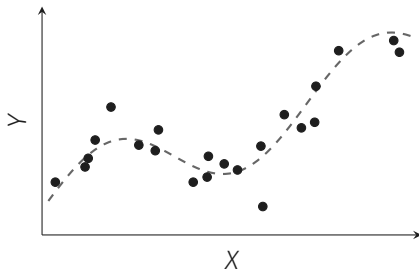


- No mundo, um **fenômeno** gera os dados:

$$Y = f(X) + \varepsilon$$

X : as entradas; Y : a resposta; ε : o ruído, o que X não explica;

O que é aprender, aqui



- dados observados
- - - f : o fenômeno (desconhecido)

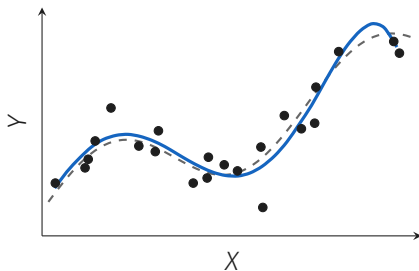
- No mundo, um **fenômeno** gera os dados:

$$Y = f(X) + \varepsilon$$

X : as entradas; Y : a resposta; ε : o ruído, o que X não explica;

- Não conhecemos f . Temos só **exemplos** (x_i, y_i) ;

O que é aprender, aqui



- dados observados
- - - f : o fenômeno (desconhecido)
- $\hat{f}$: o que aprendemos

- No mundo, um **fenômeno** gera os dados:

$$Y = f(X) + \varepsilon$$

X : as entradas; Y : a resposta; ε : o ruído, o que X não explica;

- Não conhecemos f . Temos só **exemplos** (x_i, y_i) ;
- **Aprender** é usar os exemplos para construir uma função $\hat{f}$ próxima de f ;
- O objetivo é acertar **casos novos**, e não só os exemplos usados no treino.

Aprender: três escolhas

Classe de modelos

O que o modelo consegue aprender?

A **classe de modelos**: o conjunto de funções entre as quais o treino escolhe.

Na regressão linear: todas as retas.

+

Função de perda

O que conta como errar?

A **função de perda**: uma pontuação que separa os modelos bons dos ruins.

Na regressão linear: o erro quadrático.

+

Otimização

Como achar, na classe, o modelo que erra menos?

A **otimização**: o método de busca.

Na regressão linear: uma fórmula direta.

Os algoritmos de ML combinam essas três escolhas.

Pergunta: as três escolhas

Relacione cada escolha à sua função: (1) definir o conjunto de funções entre as quais o treino escolhe; (2) medir o erro; (3) procurar o modelo que erra menos.

- a) Classe de modelos $\rightarrow$ 2; função de perda $\rightarrow$ 3; otimização $\rightarrow$ 1;
- b) Classe de modelos $\rightarrow$ 1; função de perda $\rightarrow$ 3; otimização $\rightarrow$ 2;
- c) Classe de modelos $\rightarrow$ 1; função de perda $\rightarrow$ 2; otimização $\rightarrow$ 3;
- d) Classe de modelos $\rightarrow$ 3; função de perda $\rightarrow$ 2; otimização $\rightarrow$ 1.

Pergunta: as três escolhas

Relacione cada escolha à sua função: (1) definir o conjunto de funções entre as quais o treino escolhe; (2) medir o erro; (3) procurar o modelo que erra menos.

- a) Classe de modelos $\rightarrow$ 2; função de perda $\rightarrow$ 3; otimização $\rightarrow$ 1;
- b) Classe de modelos $\rightarrow$ 1; função de perda $\rightarrow$ 3; otimização $\rightarrow$ 2;
- c) Classe de modelos $\rightarrow$ 1; função de perda $\rightarrow$ 2; otimização $\rightarrow$ 3;
- d) Classe de modelos $\rightarrow$ 3; função de perda $\rightarrow$ 2; otimização $\rightarrow$ 1.

Resposta: c. A classe de modelos diz o que pode ser aprendido; a perda mede o erro; a otimização procura, na classe, o que erra menos.

Representações

O que é uma representação

Uma representação é **outro** jeito de escrever os mesmos dados.

O mesmo número

romano XLVII $\times$ XIX = ?

arábico 47 $\times$ 19 = 893

Multiplicar: **difícil** em romanos,
fácil em arábianos.

O que é uma representação

Uma representação é **outro** jeito de escrever os mesmos dados.

O mesmo número

romano XLVII $\times$ XIX = ?

arábico 47 $\times$ 19 = 893

Multiplicar: **difícil** em romanos,
fácil em árabicos.

A mesma ocorrência

endereço R. Conde de Bonfim, 500, Tijuca

coordenadas (−22,92; −43,23)

“Ficou a menos de 1 km do batalhão?”:
fácil com as coordenadas.

“Para onde mandar a viatura?”:
fácil com o endereço.

O que é uma representação

Uma representação é **outro jeito de escrever os mesmos dados**.

O mesmo número

romano	$\text{XLVII} \times \text{XIX} = ?$
arábico	$47 \times 19 = 893$

Multiplicar: **difícil** em romanos,
fácil em árabicos.

A mesma ocorrência

endereço	R. Conde de Bonfim, 500, Tijuca
coordenadas	$(-22,92; -43,23)$

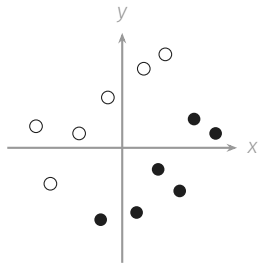
“Ficou a menos de 1 km do batalhão?”:
fácil com as coordenadas.

“Para onde mandar a viatura?”:
fácil com o endereço.

A melhor representação depende da tarefa.

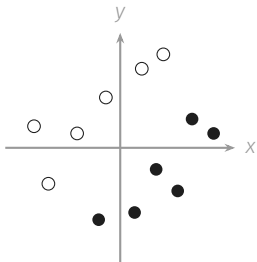
Modelos de ML procuram a **representação** que facilita a tarefa.

Um exemplo: pontos pretos e brancos

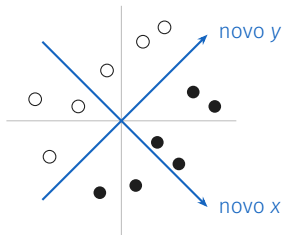


1. os dados

Um exemplo: pontos pretos e brancos

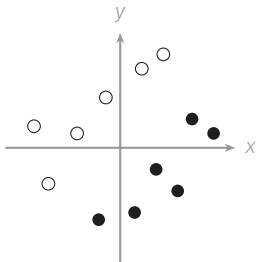


1. os dados

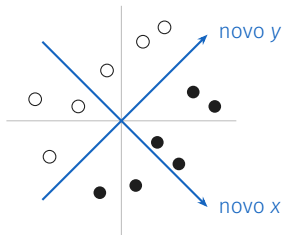


2. mudar as coordenadas

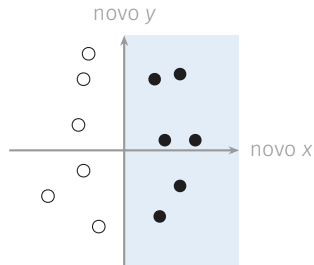
Um exemplo: pontos pretos e brancos



1. os dados

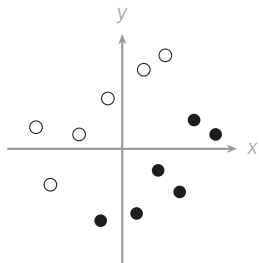


2. mudar as coordenadas

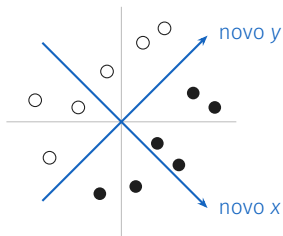


3. a nova representação

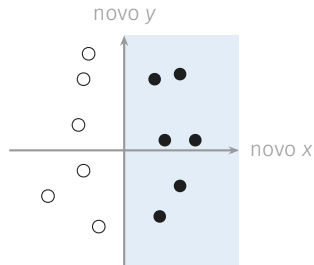
Um exemplo: pontos pretos e brancos



1. os dados



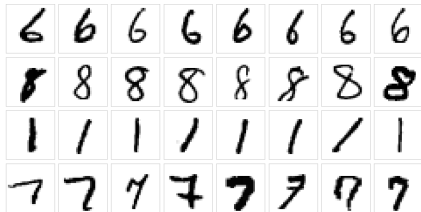
2. mudar as coordenadas



3. a nova representação

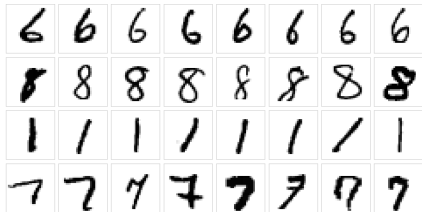
Na nova representação, a regra fica simples: **os pontos pretos têm novo $x > 0$.**

E se fossem dígitos escritos à mão?



- No exemplo anterior, **nós** escolhemos a mudança de coordenadas;
- Como separar um 6 de um 8, ou um 1 de um 7, em qualquer caligrafia?

E se fossem dígitos escritos à mão?



- No exemplo anterior, **nós** escolhemos a mudança de coordenadas;
- Como separar um 6 de um 8, ou um 1 de um 7, em qualquer caligrafia?
- Regras como “número de laços fechados” funcionam em parte;
- Cada caligrafia nova quebra alguma regra e exige regras novas, que interagem com as antigas.

E se o computador encontrasse a representação?

- Nos pontos pretos e brancos, **nós** encontramos a mudança de coordenadas. Nos dígitos, ninguém consegue escrever as regras;

E se o computador encontrasse a representação?

- Nos pontos pretos e brancos, **nós** encontramos a mudança de coordenadas. Nos dígitos, ninguém consegue escrever as regras;
- A ideia: deixar o computador **testar transformações da imagem** e ficar com as que mais acertam os dígitos;
- O que guia a escolha é a **perda**: o quanto a classificação errou nos exemplos de treino.

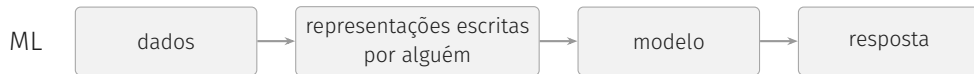
E se o computador encontrasse a representação?

- Nos pontos pretos e brancos, **nós** encontramos a mudança de coordenadas. Nos dígitos, ninguém consegue escrever as regras;
- A ideia: deixar o computador **testar transformações da imagem** e ficar com as que mais acertam os dígitos;
- O que guia a escolha é a **perda**: o quanto a classificação errou nos exemplos de treino.

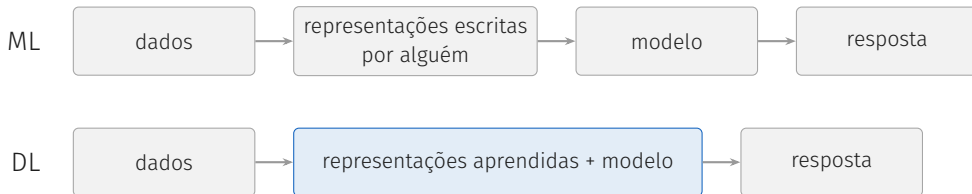
Aprender, de novo

Aprender continua sendo construir $\hat{f}$ próxima de f . No deep learning, $\hat{f}$ começa transformando a entrada, e essa transformação **também é aprendida**.

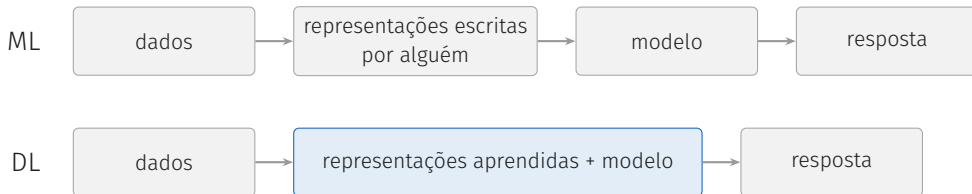
ML e DL: quem escreve as representações



ML e DL: quem escreve as representações



ML e DL: quem escreve as representações



- No ML, quando as colunas recebidas não bastam, alguém escolhe ou constrói representações úteis. No deep learning, essas representações também são aprendidas a partir dos exemplos;
- O deep learning segue as mesmas etapas do projeto de ML: **definir o problema, preparar os dados, dividir treino e teste, pré-processar, treinar, prever e avaliar**;
- Imagem, som e texto: deep learning. Tabelas: árvores e florestas costumam bastar.

Dados estruturados e não estruturados

Estruturados

área (m ²)	quartos	bairro	preço (R\$ mil)
60	2	Tijuca	480
35	1	Centro	210
110	3	Botafoogo	770
⋮	⋮	⋮	⋮

Uma tabela. Cada coluna tem um significado definido.

Não estruturados



áudio



imagem

"Por volta das 22h, o solicitante informou que dois homens em uma moto..."

texto

Dados estruturados e não estruturados

Estruturados

área (m ²)	quartos	bairro	preço (R\$ mil)
60	2	Tijuca	480
35	1	Centro	210
110	3	Botafoogo	770
⋮	⋮	⋮	⋮

Uma tabela. Cada coluna tem um **significado definido**.

- No áudio, na imagem e no texto, o computador recebe números (amostras de som, pixels, letras). Nenhum deles, sozinho, tem significado;
- O deep learning trouxe o maior ganho nos dados **não estruturados**. Nas tabelas, árvores e florestas continuam competitivas.

Não estruturados



áudio



imagem

"Por volta das 22h, o solicitante informou que dois homens em uma moto..."

texto

Pergunta: rede ou regressão linear?

Para estimar o custo de manutenção de viaturas, há 200 registros com a idade e a quilometragem de cada uma. O custo cresce de forma quase reta com as duas. A chefia quer entender o que pesa no custo. Qual escolha inicial faz mais sentido?

- a) Uma rede neural profunda, porque redes criam as próprias colunas e sempre vencem;
- b) Uma rede neural grande, para não deixar nenhuma curva escondida de fora;
- c) Regressão linear como primeira tentativa e referência; uma rede só entra se for melhor que ela;
- d) Nenhum modelo: com só duas colunas, ML não ajuda.

Pergunta: rede ou regressão linear?

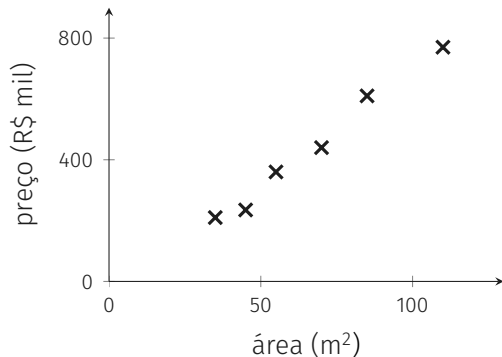
Para estimar o custo de manutenção de viaturas, há 200 registros com a idade e a quilometragem de cada uma. O custo cresce de forma quase reta com as duas. A chefia quer entender o que pesa no custo. Qual escolha inicial faz mais sentido?

- a) Uma rede neural profunda, porque redes criam as próprias colunas e sempre vencem;
- b) Uma rede neural grande, para não deixar nenhuma curva escondida de fora;
- c) Regressão linear como primeira tentativa e referência; uma rede só entra se for melhor que ela;
- d) Nenhum modelo: com só duas colunas, ML não ajuda.

Resposta: c. Poucos dados, relação quase reta e necessidade de explicar: o modelo linear é o ponto de partida e a referência para qualquer rede.

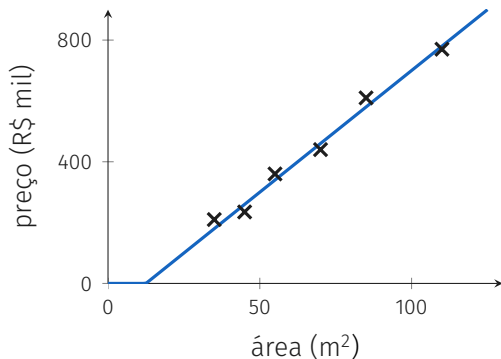
A rede neural

Prever o preço de um apartamento



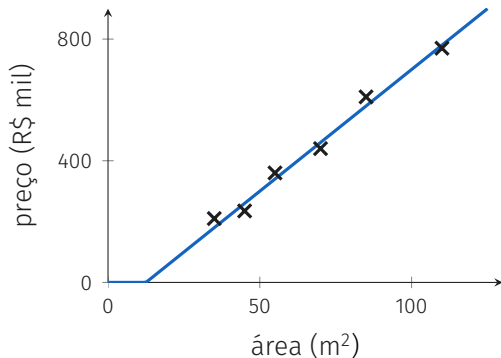
- Seis apartamentos: a área e o preço de cada um;

Prever o preço de um apartamento



- Seis apartamentos: a área e o preço de cada um;
- Uma reta passa perto dos pontos. Mas, para apartamentos muito pequenos, a reta dá **preço negativo**;
- Então a reta é **cortada em zero**;

Prever o preço de um apartamento

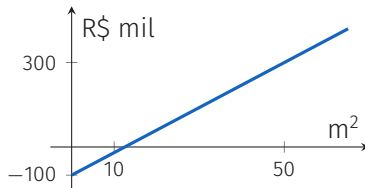


- Seis apartamentos: a área e o preço de cada um;
- Uma reta passa perto dos pontos. Mas, para apartamentos muito pequenos, a reta dá **preço negativo**;
- Então a reta é **cortada em zero**;
- Os próximos slides montam essa função, peça por peça.

Função afim

$$f(x) = ax + b$$

a : quanto f sobe quando x aumenta 1 b : o valor de f em $x = 0$ o gráfico é uma **reta**



Função afim

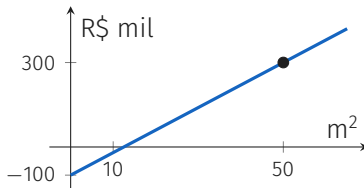
$$f(x) = ax + b$$

a : quanto f sobe quando x aumenta 1 b : o valor de f em $x = 0$ o gráfico é uma **reta**

O apartamento (preço em R\$ mil, área em m^2):

$$f(\text{área}) = 8 \cdot \text{área} - 100$$

- 50 m^2 : $8 \cdot 50 - 100 = 300$, ou seja, R\$ 300 mil;



Função afim

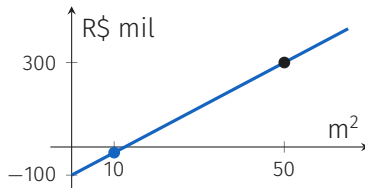
$$f(x) = ax + b$$

a : quanto f sobe quando x aumenta 1 b : o valor de f em $x = 0$ o gráfico é uma **reta**

O apartamento (preço em R\$ mil, área em m^2):

$$f(\text{área}) = 8 \cdot \text{área} - 100$$

- 50 m^2 : $8 \cdot 50 - 100 = 300$, ou seja, R\$ 300 mil;
- 10 m^2 : $8 \cdot 10 - 100 = -20$. Preço negativo.



Função afim com várias entradas

$$f(x_1, x_2) = w_1x_1 + w_2x_2 + b$$

O apartamento, agora com área e quartos:

$$f(\text{área}, \text{quartos}) = 8 \cdot \text{área} + 50 \cdot \text{quartos} - 100$$

60 m² e 2 quartos: $8 \cdot 60 + 50 \cdot 2 - 100 = 480$, ou seja, R\$ 480 mil.

Função afim com várias entradas

$$f(x_1, x_2) = w_1x_1 + w_2x_2 + b$$

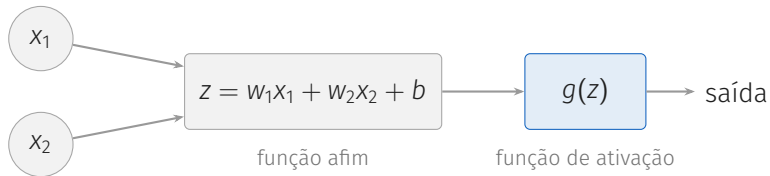
O apartamento, agora com área e quartos:

$$f(\text{área}, \text{quartos}) = 8 \cdot \text{área} + 50 \cdot \text{quartos} - 100$$

60 m² e 2 quartos: $8 \cdot 60 + 50 \cdot 2 - 100 = 480$, ou seja, R\$ 480 mil.

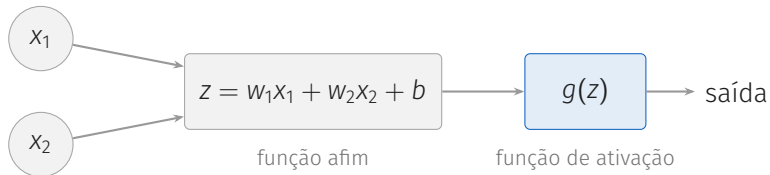
- Em ML, os números w_1, w_2 são os **pesos**, e b é o **bias**;
- São esses números que o modelo **aprende** com os exemplos.

O neurônio: função afim + função de ativação



Um **neurônio** é uma função afim seguida de uma **função de ativação** g .

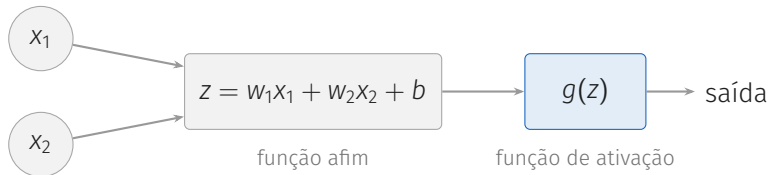
O neurônio: função afim + função de ativação



Um **neurônio** é uma função afim seguida de uma **função de ativação** g .

- Com a **ReLU**, $g(z) = \max(0, z)$, que zera o negativo: 60 m² e 2 quartos dão $z = 480$ e saída 480; 10 m² e nenhum quarto dão $z = -20$ e saída 0;
- A reta cortada em zero do primeiro slide é um neurônio com uma entrada (a área) e ReLU;

O neurônio: função afim + função de ativação



Um **neurônio** é uma função afim seguida de uma **função de ativação** g .

- Com a **ReLU**, $g(z) = \max(0, z)$, que zera o negativo: 60 m² e 2 quartos dão $z = 480$ e saída 480; 10 m² e nenhum quarto dão $z = -20$ e saída 0;
- A reta cortada em zero do primeiro slide é um neurônio com uma entrada (a área) e ReLU;
- Com a **sigmoide**, que espreme a saída entre 0 e 1, o neurônio responde “sim ou não”: é a **regressão logística**.

As duas funções de ativação de hoje

- A função de ativação é aplicada depois da função afim;
- O gráfico dela **não é uma reta**. Isso faz diferença (próximos slides).

z	-2	-0,5	0	0,5	2
ReLU: $\max(0, z)$, “zera o negativo”	0	0	0	0,5	2
Sigmoide: “espreme entre 0 e 1”	0,12	0,38	0,5	0,62	0,88

Pergunta: o neurônio

O que é um neurônio artificial?

- a) O método que procura o modelo que erra menos;
- b) Uma função afim das entradas seguida de uma função de ativação;
- c) A função que mede o erro entre a previsão e a resposta certa;
- d) Uma linha da tabela de dados.

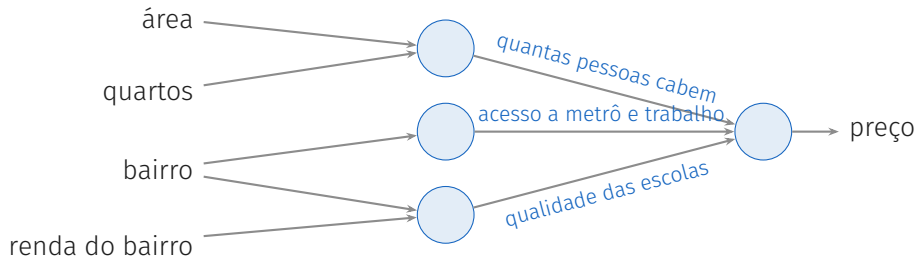
Pergunta: o neurônio

O que é um neurônio artificial?

- a) O método que procura o modelo que erra menos;
- b) Uma função afim das entradas seguida de uma função de ativação;
- c) A função que mede o erro entre a previsão e a resposta certa;
- d) Uma linha da tabela de dados.

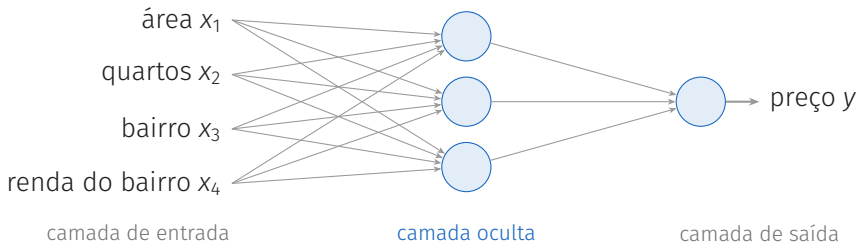
Resposta: b. Soma das entradas multiplicadas pelos pesos, mais o bias, e depois a função de ativação. As outras alternativas descrevem a otimização (a), a perda (c) e um exemplo (d).

Vários neurônios: uma camada



- Cada neurônio calcula um novo valor a partir das entradas;
- Neurônios que trabalham na mesma etapa, lado a lado, formam uma **camada**;
- O último neurônio combina os valores produzidos pela camada anterior e dá o preço.

A rede neural



- **Camada de entrada:** os dados (x). **Camada oculta:** a do meio. **Camada de saída:** a resposta (y);
- Cada entrada se liga a todos os neurônios escondidos. A rede decide sozinha **o que cada um calcula**: por isso, oculta;
- Uma **rede neural** é uma sequência de camadas.

Pergunta: lendo uma rede

Uma rede recebe 4 valores (área, quartos, bairro, renda do bairro), tem uma camada oculta com 3 neurônios e ReLU, e uma camada de saída com 1 neurônio.

Cada neurônio da camada oculta recebe quantas entradas?

- a) 1;
- b) 3;
- c) 4;
- d) 12.

Pergunta: lendo uma rede

Uma rede recebe 4 valores (área, quartos, bairro, renda do bairro), tem uma camada oculta com 3 neurônios e ReLU, e uma camada de saída com 1 neurônio.

Cada neurônio da camada oculta recebe quantas entradas?

- a) 1;
- b) 3;
- c) 4;
- d) 12.

Resposta: c. Cada entrada se liga a todos os neurônios da camada oculta: cada neurônio recebe as 4 entradas. O 12 é o número de ligações da camada inteira (4×3).

Sem a função de ativação, empilhar não adianta

Duas funções afins, uma depois da outra:

$$f(x) = 2x + 1 \quad g(u) = 3u - 2$$

Sem a função de ativação, empilhar não adianta

Duas funções afins, uma depois da outra:

$$f(x) = 2x + 1 \quad g(u) = 3u - 2$$

$$g(f(x)) = 3(2x + 1) - 2 = 6x + 1$$

Sem a função de ativação, empilhar não adianta

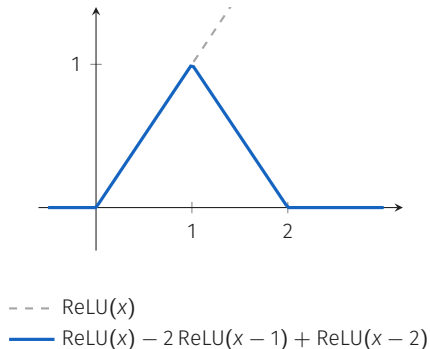
Duas funções afins, uma depois da outra:

$$f(x) = 2x + 1 \quad g(u) = 3u - 2$$

$$g(f(x)) = 3(2x + 1) - 2 = 6x + 1$$

- O resultado é outra função afim: **uma reta**;
- Empilhar 100 camadas só de funções afins dá uma função afim: o mesmo que uma regressão linear;
- É a função de ativação entre as camadas que permite à rede desenhar **curvas**.

Com a função de ativação: curvas



- Três neurônios com ReLU, somados com pesos 1, -2 e 1, desenharam um “pico”;
- Nenhuma função afim desenha isso;
- Com mais neurônios, a rede desenha formas mais complicadas. Vamos ver isso no Playground.

Pergunta: sem a função de ativação

Uma rede tem 10 camadas, todas só com funções afins, sem função de ativação. O que ela consegue desenhar?

- a) Curvas: cada camada acrescenta uma curva nova;
- b) Curvas, desde que cada camada tenha um número diferente de neurônios;
- c) Só retas: as 10 camadas equivalem a uma única função afim;
- d) Nada: sem função de ativação, a rede não consegue treinar.

Pergunta: sem a função de ativação

Uma rede tem 10 camadas, todas só com funções afins, sem função de ativação. O que ela consegue desenhar?

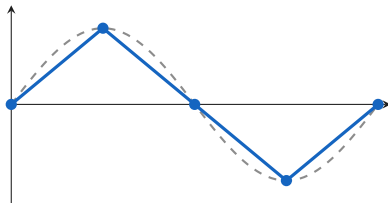
- a) Curvas: cada camada acrescenta uma curva nova;
- b) Curvas, desde que cada camada tenha um número diferente de neurônios;
- c) **Só retas: as 10 camadas equivalem a uma única função afim;**
- d) Nada: sem função de ativação, a rede não consegue treinar.

Resposta: c. Função afim de função afim é função afim. A alternativa (d) é o erro comum: a rede treina normalmente, mas só desenha retas.

Teorema da aproximação universal

Em palavras

Uma rede com **uma camada oculta** e uma função de ativação como a ReLU consegue chegar tão perto quanto se queira de **qualquer função contínua**, desde que tenha **neurônios suficientes**.



--- a função que queremos

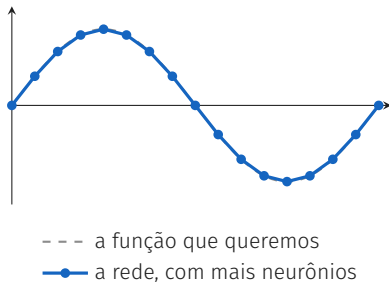
—●— a rede, com poucos neurônios

- Cada neurônio com ReLU acrescenta uma “dobra” na reta;
- Somando dobras, a rede desenha segmentos que seguem a curva;

Teorema da aproximação universal

Em palavras

Uma rede com **uma camada oculta** e uma função de ativação como a ReLU consegue chegar tão perto quanto se queira de **qualquer função contínua**, desde que tenha **neurônios suficientes**.



- Cada neurônio com ReLU acrescenta uma “dobra” na reta;
- Somando dobras, a rede desenha segmentos que seguem a curva;
- Com mais neurônios, os segmentos ficam menores e a rede fica mais perto da curva.

O que o teorema não garante

O teorema diz que a rede **existe**. Ele não diz:

- **quantos neurônios** são necessários: podem ser muitos;
- **como encontrar os pesos**: o treino pode não achar a rede que o teorema promete;
- se a rede vai **acertar casos novos**: chegar perto de f nos exemplos de treino não basta.

O que o teorema não garante

O teorema diz que a rede **existe**. Ele não diz:

- **quantos neurônios** são necessários: podem ser muitos;
- **como encontrar os pesos**: o treino pode não achar a rede que o teorema promete;
- se a rede vai **acertar casos novos**: chegar perto de f nos exemplos de treino não basta.

Na prática, várias camadas em sequência costumam precisar de **menos neurônios** do que uma camada só. Daí o *deep*.

O que quer dizer “deep”

- A rede monta a função $\hat{f}$ camada por camada: a saída de uma camada é a entrada da seguinte.

$$\hat{f}(\mathbf{x}) = f_3(f_2(f_1(\mathbf{x})))$$

- A **profundidade** é o número de camadas. **Deep** (profundo) quer dizer muitas camadas. Hoje, dezenas ou centenas;

O que quer dizer “deep”

- A rede monta a função $\hat{f}$ camada por camada: a saída de uma camada é a entrada da seguinte.

$$\hat{f}(\mathbf{x}) = f_3(f_2(f_1(\mathbf{x})))$$

- A **profundidade** é o número de camadas. **Deep** (profundo) quer dizer muitas camadas. Hoje, dezenas ou centenas;
- A regressão logística e a árvore decidem a partir das colunas que recebem. A rede monta, camada por camada, as “colunas” de que precisa para decidir;

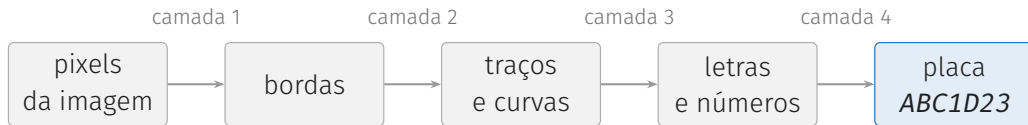
O que quer dizer “deep”

- A rede monta a função $\hat{f}$ camada por camada: a saída de uma camada é a entrada da seguinte.

$$\hat{f}(\mathbf{x}) = f_3(f_2(f_1(\mathbf{x})))$$

- A **profundidade** é o número de camadas. **Deep** (profundo) quer dizer muitas camadas. Hoje, dezenas ou centenas;
- A regressão logística e a árvore decidem a partir das colunas que recebem. A rede monta, camada por camada, as “colunas” de que precisa para decidir;
- O nome “rede neural” vem da biologia, mas a rede **não** é um modelo do cérebro.

Um exemplo: ler uma placa



- Cada camada parte do que a anterior encontrou e monta algo **mais abstrato**;
- Ninguém diz a cada camada o que procurar: isso é **aprendido** com os exemplos.

Uma breve história

De 1943 aos LLMs

neurônio artificial
(McCulloch e Pitts)



1943

De 1943 aos LLMs

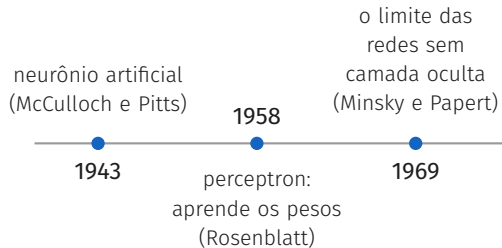
neurônio artificial
(McCulloch e Pitts)

1943

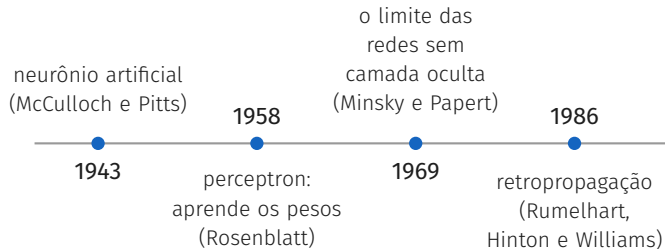
1958

perceptron:
aprende os pesos
(Rosenblatt)

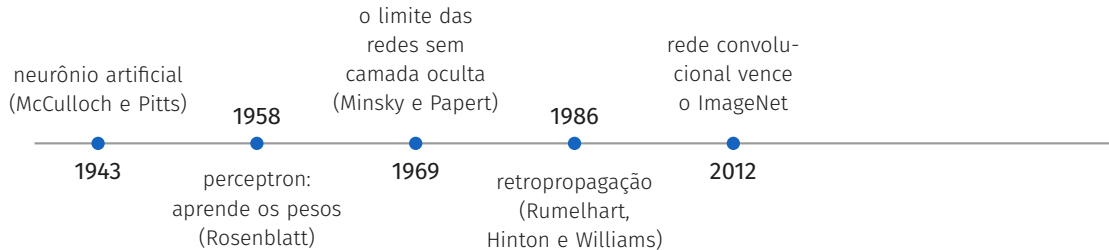
De 1943 aos LLMs



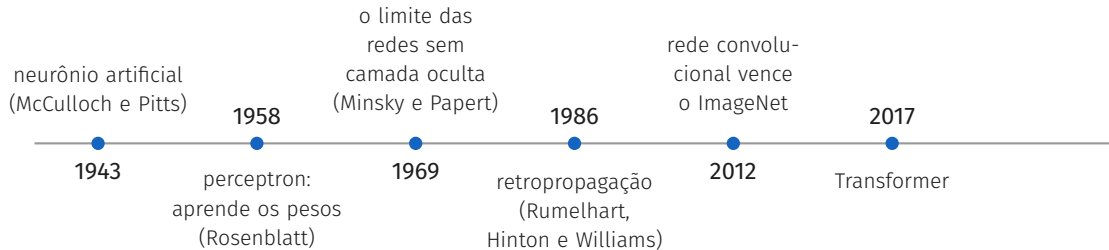
De 1943 aos LLMs



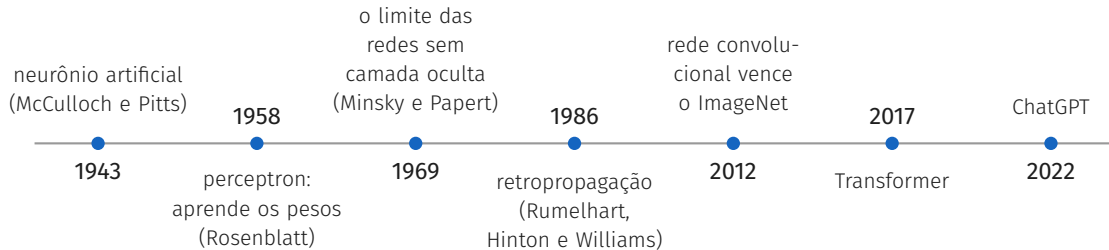
De 1943 aos LLMs



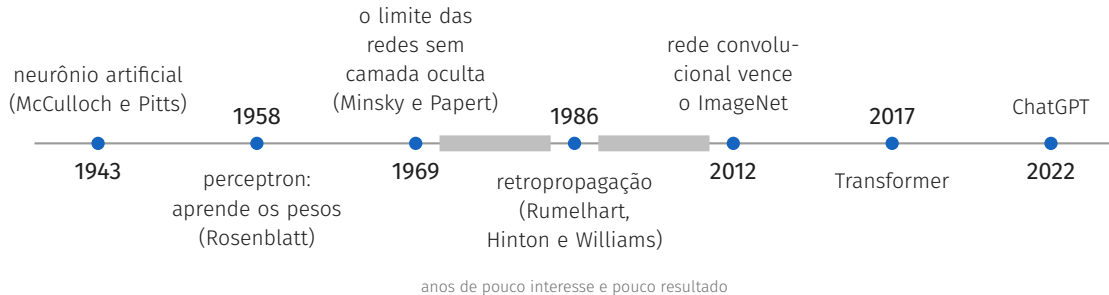
De 1943 aos LLMs



De 1943 aos LLMs

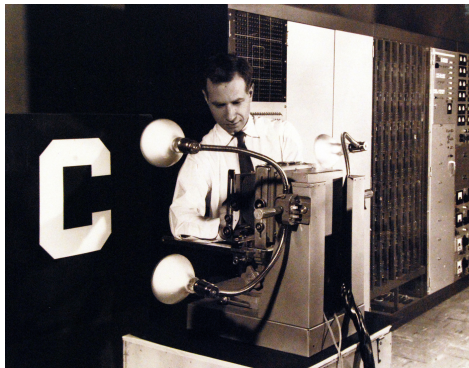


De 1943 aos LLMs



1958: o perceptron aprende os pesos

- McCulloch e Pitts (1943): um “neurônio” que soma entradas com pesos e dispara ou não;
- Rosenblatt (1958): o perceptron, **um neurônio só**. Os pesos **não precisam ser escritos à mão**: o perceptron os ajusta a partir de exemplos;
- Uma máquina física, com uma câmera de 20×20 fotocélulas, que aprendia a separar letras e formas.



1969: o XOR

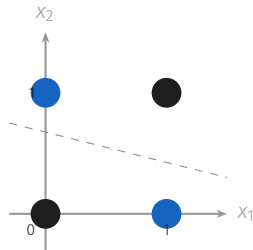
x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

A saída é 1 quando **exatamente uma** das entradas é 1.

1969: o XOR

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

A saída é 1 quando **exatamente uma** das entradas é 1.



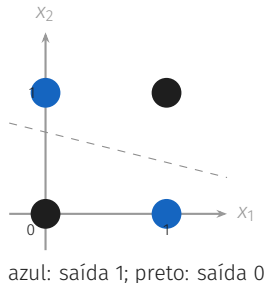
azul: saída 1; preto: saída 0

- Nenhuma reta separa os pontos azuis dos pretos: uma reta acerta no máximo 3 dos 4 casos;

1969: o XOR

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

A saída é 1 quando **exatamente uma** das entradas é 1.

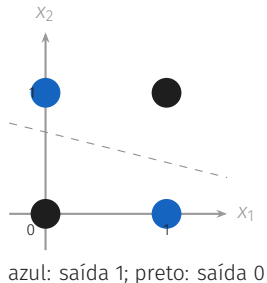


- Nenhuma reta separa os pontos azuis dos pretos: uma reta acerta no máximo 3 dos 4 casos;
- A regressão logística e o perceptron traçam **uma reta**. Os dois falham no XOR;
- Minsky e Papert (1969) mostraram esse limite; o interesse e o dinheiro sumiram por anos;

1969: o XOR

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

A saída é 1 quando **exatamente uma** das entradas é 1.



- Nenhuma reta separa os pontos azuis dos pretos: uma reta acerta no máximo 3 dos 4 casos;
- A regressão logística e o perceptron traçam **uma reta**. Os dois falham no XOR;
- Minsky e Papert (1969) mostraram esse limite; o interesse e o dinheiro sumiram por anos;
- [Hoje, no Playground](#): o que resolve isso.

- **Frank Rosenblatt, 1958**, na demonstração do perceptron (relato do New York Times): *“Perceptrons futuros vão reconhecer pessoas e dizer seus nomes, e traduzir na hora a fala de uma língua para a fala e a escrita de outra.”* Hoje, as duas coisas existem, feitas com redes neurais;

- **Frank Rosenblatt, 1958**, na demonstração do perceptron (relato do New York Times): *“Perceptrons futuros vão reconhecer pessoas e dizer seus nomes, e traduzir na hora a fala de uma língua para a fala e a escrita de outra.”* Hoje, as duas coisas existem, feitas com redes neurais;
- **Marvin Minsky, 1970**: *“Em três a oito anos teremos uma máquina com a inteligência geral de um ser humano médio.”* A previsão falhou e o financiamento secou: o **primeiro inverno da IA**, nos anos 1970;
- Nos anos 1980, os sistemas especialistas (regras escritas à mão) atraíram mais de 1 bilhão de dólares por ano. No início dos anos 1990, eram caros de manter e limitados: o **segundo inverno**.

Anos 1980 e 1990: retropropagação e a primeira aplicação

- **Meados dos anos 1980:** a **retropropagação** permite treinar redes com camadas ocultas;
- **1989:** Yann LeCun, no Bell Labs, junta redes convolucionais e retropropagação para ler dígitos manuscritos. A rede (LeNet) leu os CEPs das cartas do correio americano nos anos 1990;

Anos 1980 e 1990: retropropagação e a primeira aplicação

- **Meados dos anos 1980:** a **retropropagação** permite treinar redes com camadas ocultas;
- **1989:** Yann LeCun, no Bell Labs, junta redes convolucionais e retropropagação para ler dígitos manuscritos. A rede (LeNet) leu os CEPs das cartas do correio americano nos anos 1990;
- **Anos 1990 e 2000:** SVMs e, depois, árvores e florestas superaram as redes. Esses modelos usam as colunas que recebem: para imagens, alguém precisa calcular as colunas à mão (por exemplo, contar os laços fechados de cada dígito).

2012: o ImageNet

- A competição: classificar fotos coloridas em **1.000 categorias**, treinando com **1,4 milhão** de imagens;
- A métrica: **acerto top-5**, ou seja, a resposta certa está entre os 5 palpites do modelo;

Ano	Modelo vencedor	Acerto top-5
2011	visão computacional clássica	74,3%
2012	rede convolucional (Krizhevsky, orientado por Hinton)	83,6%
2015	rede convolucional	96,4%

Desde 2012, toda edição foi vencida por redes convolucionais (Aulas 3 e 4).

- **2017:** o **Transformer** (“Attention is all you need”): a arquitetura por trás do GPT e do BERT (Aula 7);
- **2022:** o **ChatGPT** chega ao público;
- **2018:** prêmio Turing para Hinton, LeCun e Bengio; **2024:** Nobel de Física para Hopfield e Hinton.

Por que agora?

As ideias principais existem desde os anos 1980. O que mudou depois de 2010:

Dados

A internet gerou milhões de fotos, textos e áudios. Surgiram bases públicas rotuladas, como o ImageNet (1,4 milhão de imagens).

Poder computacional

As **GPUs**, placas de vídeo criadas para jogos, fazem milhares de multiplicações ao mesmo tempo. Treinar uma rede é, em grande parte, multiplicar.

Algoritmos

Mudanças que permitem treinar redes com muitas camadas: a ReLU e formas melhores de iniciar e de ajustar os pesos.

TensorFlow Playground

Abram o playground.tensorflow.org

- **Sem camada oculta:** não resolve. Uma reta, igual à regressão logística: o limite de 1969;
- **Com uma camada oculta** de poucos neurônios: resolve. A camada transformou os dados para uma representação em que os 1 e os 0 ficam mais fáceis de separar;
- **A espiral** pede mais neurônios, ou mais camadas.

- **Sem camada oculta:** não resolve. Uma reta, igual à regressão logística: o limite de 1969;
- **Com uma camada oculta** de poucos neurônios: resolve. A camada transformou os dados para uma representação em que os 1 e os 0 ficam mais fáceis de separar;
- **A espiral** pede mais neurônios, ou mais camadas.

A camada oculta **aumenta as formas** que o modelo consegue desenhar.

Como a rede aprende

A rede é uma composição de funções

$$\hat{f}(\mathbf{x}) = \text{sigmoide}\left(\text{afim}_2\left(\underbrace{\text{ReLU}(\text{afim}_1(\mathbf{x}))}_{\text{saída da camada oculta: nova representação}}\right)\right)$$

- A rede transforma os dados em etapas;
- A saída de uma etapa vira a entrada da próxima;
- Os pesos e os bias de todas as funções afins são os **parâmetros**: o que a rede aprende.

Treinar é achar os parâmetros

- O exemplo do salário, no ML: $\text{salário} = a + b \cdot \text{anos}$. Treinar era achar 2 números;

Treinar é achar os parâmetros

- O exemplo do salário, no ML: $\text{salário} = a + b \cdot \text{anos}$. Treinar era achar 2 números;
- E se fossem 100 mil?

Treinar é achar os parâmetros

- O exemplo do salário, no ML: $\text{salário} = a + b \cdot \text{anos}$. Treinar era achar 2 **números**;
- E se fossem 100 mil?
- Treinar uma rede é escolher os pesos que **minimizam a perda média** nos exemplos de treino:

$$\min_{\text{pesos, bias}} \frac{1}{N} \sum_{i=1}^N \text{perda}(\hat{f}(\mathbf{x}_i), y_i)$$

A perda: o quanto a rede errou

Erro quadrático médio (números)

real	previsto	erro ²
1	0,9	0,01
0	0,2	0,04
1	0,6	0,16
média		0,07

A perda: o quanto a rede errou

Erro quadrático médio (números)

real	previsto	erro ²
1	0,9	0,01
0	0,2	0,04
1	0,6	0,16
média		0,07

Entropia cruzada (classes)

p : a chance que a rede deu à resposta certa.

p	perda = $-\ln p$
0,9	0,11
0,5	0,69
0,1	2,30

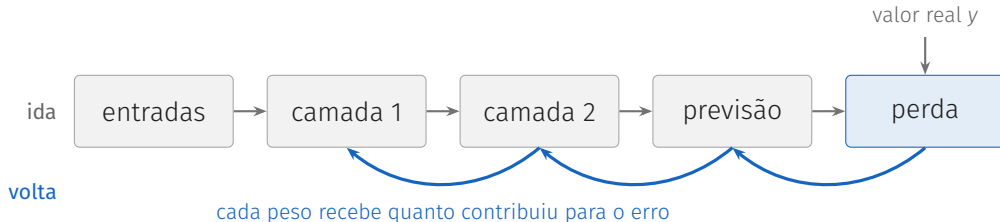
“O quanto a rede **se surpreendeu** com a resposta certa.”

Ida e volta: forward pass e backward pass



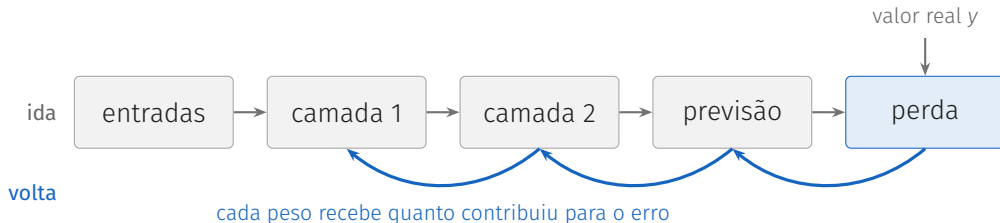
- **Ida** (*forward pass*): os dados atravessam a rede, sai a previsão, e a perda mede o erro;

Ida e volta: forward pass e backward pass



- **Ida** (*forward pass*): os dados atravessam a rede, sai a previsão, e a perda mede o erro;
- **Volta** (*backward pass*): o erro é devolvido, camada por camada, até cada peso. Esse cálculo se chama **retropropagação**;

Ida e volta: forward pass e backward pass



- **Ida** (*forward pass*): os dados atravessam a rede, sai a previsão, e a perda mede o erro;
- **Volta** (*backward pass*): o erro é devolvido, camada por camada, até cada peso. Esse cálculo se chama **retropropagação**;
- Depois da volta, os pesos são ajustados para diminuir a perda, e começa outra ida.

Pergunta: ida, volta ou as duas?

Uma rede já treinada recebe a foto de uma placa e devolve **ABC1D23**. Nesse momento, ela faz:

- a) Ida e volta, ajustando os pesos com a foto nova;
- b) Só a ida: uma previsão;
- c) Só a volta: calcula o erro da foto;
- d) Nenhuma das duas: só consulta uma tabela de placas.

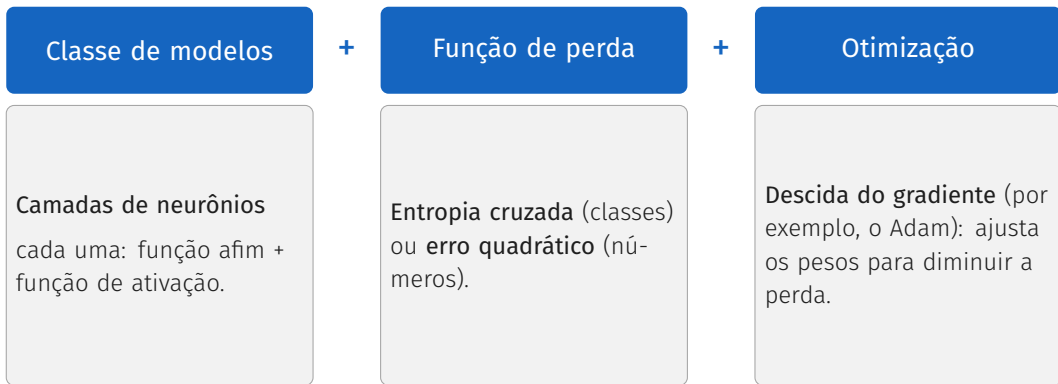
Pergunta: ida, volta ou as duas?

Uma rede já treinada recebe a foto de uma placa e devolve **ABC1D23**. Nesse momento, ela faz:

- a) Ida e volta, ajustando os pesos com a foto nova;
- b) **Só a ida: uma previsão;**
- c) Só a volta: calcula o erro da foto;
- d) Nenhuma das duas: só consulta uma tabela de placas.

Resposta: b. Depois do treino, usar a rede é só a ida. A volta precisa da resposta certa, que a rede não tem para uma foto nova.

Juntando tudo: a rede neural nas três caixas



As arquiteturas das próximas aulas mudam uma ou mais dessas caixas.

Os modelos do ML, nas três caixas

Método	Classe de modelos (o que ele consegue aprender)	Função de perda	Otimização
kNN	respostas dadas pelos k exemplos mais parecidos	não há (memoriza)	nenhuma: guarda os exemplos
Regressão linear	funções afins (retas)	erro quadrático	fórmula fechada (mínimos quadrados)
Regressão logística	função afim seguida da sigmoide	entropia cruzada	iterativa (L-BFGS, padrão do scikit-learn)
Árvore	regras de sim ou não, em sequência	impureza (Gini ou entropia)	busca gulosa, um nó por vez
k-means	divisões dos dados em k grupos, cada um em volta de um centro	distância aos centros	alterna: atribuir pontos, mover centros
Rede neural	camadas de neurônios em sequência	entropia cruzada ou erro quadrático	descida do gradiente (Adam)

Pergunta: o que o treino aprende?

Qual destes o treino da rede aprende a partir dos exemplos?

- a) O número de camadas;
- b) A função de ativação de cada camada;
- c) Os pesos e os bias;
- d) O número de neurônios de cada camada.

Pergunta: o que o treino aprende?

Qual destes o treino da rede aprende a partir dos exemplos?

- a) O número de camadas;
- b) A função de ativação de cada camada;
- c) Os pesos e os bias;
- d) O número de neurônios de cada camada.

Resposta: c. Os pesos e os bias são os parâmetros: o treino os ajusta. Camadas, neurônios e funções de ativação são escolhidos por vocês, antes do treino.

Keras

- Nas próximas aulas, vamos montar e treinar redes com o **Keras**, uma biblioteca de Python.

Fechamento

1. Um neurônio é uma função afim seguida de uma função de ativação, e a rede é uma **composição** de camadas de neurônios;

As 3 frases para levar

1. Um neurônio é uma função afim seguida de uma função de ativação, e a rede é uma **composição** de camadas de neurônios;
2. Aprender é **ajustar os pesos** para diminuir a perda;

As 3 frases para levar

1. Um neurônio é uma função afim seguida de uma função de ativação, e a rede é uma **composição** de camadas de neurônios;
2. Aprender é **ajustar os pesos** para diminuir a perda;
3. Treinar um modelo envolve três escolhas: **a classe de modelos, a função de perda e a otimização.**

O vocabulário de hoje

Função	regra que associa a cada entrada uma saída
Aprender	usar exemplos para construir uma função $\hat{f}$ que acerte casos novos
Representação	outro jeito de codificar os mesmos dados
Função afim	$w_1x_1 + w_2x_2 + b$; o gráfico é uma reta
Peso	o número que multiplica cada entrada
Bias	o número somado no fim da função afim
Função de ativação	aplicada depois da função afim: ReLU zera o negativo; sigmoide espreme entre 0 e 1

Neurônio	função afim seguida de função de ativação
Camada	neurônios lado a lado, na mesma etapa
Entrada, oculta, saída	a camada dos dados; as do meio; a da resposta
Rede neural	camadas em sequência
Profundidade	o número de camadas
Perda	o quanto a rede errou
Parâmetros	os pesos e os bias; o treino aprende

14

Por que uma rede, e não o modelo de ML de vocês? Ela bateu a referência **e** o melhor modelo de ML?

Referências: livros e cursos

- CHOLLET, F. *Deep Learning with Python*. 2. ed. Manning, 2021.
- PRINCE, S. J. D. *Understanding Deep Learning*. MIT Press, 2023.
- JAMES, G.; WITTEN, D.; HASTIE, T.; TIBSHIRANI, R. *An Introduction to Statistical Learning*. 2. ed. Springer, 2021.
- NG, A. *Neural Networks and Deep Learning* (Deep Learning Specialization, curso 1). DeepLearning.AI, Coursera.
- ISOLA, P. 6.7960 *Deep Learning*, notas de aula. MIT.
- SMILKOV, D.; CARTER, S. *TensorFlow Playground*. playground.tensorflow.org.

Referências: artigos e fontes históricas

- McCULLOCH, W.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 1943.
- ROSENBLATT, F. The perceptron. *Psychological Review*, 1958.
- New Navy device learns by doing. *The New York Times*, 8 jul. 1958.
- MINSKY, M.; PAPERT, S. *Perceptrons*. MIT Press, 1969.
- RUMELHART, D.; HINTON, G.; WILLIAMS, R. Learning representations by back-propagating errors. *Nature*, 1986.
- LeCUN, Y. et al. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1989.
- DOMINGOS, P. A few useful things to know about machine learning. *Communications of the ACM*, 2012.
- KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. ImageNet classification with deep convolutional neural networks. NeurIPS, 2012.
- VASWANI, A. et al. Attention is all you need. NeurIPS, 2017.